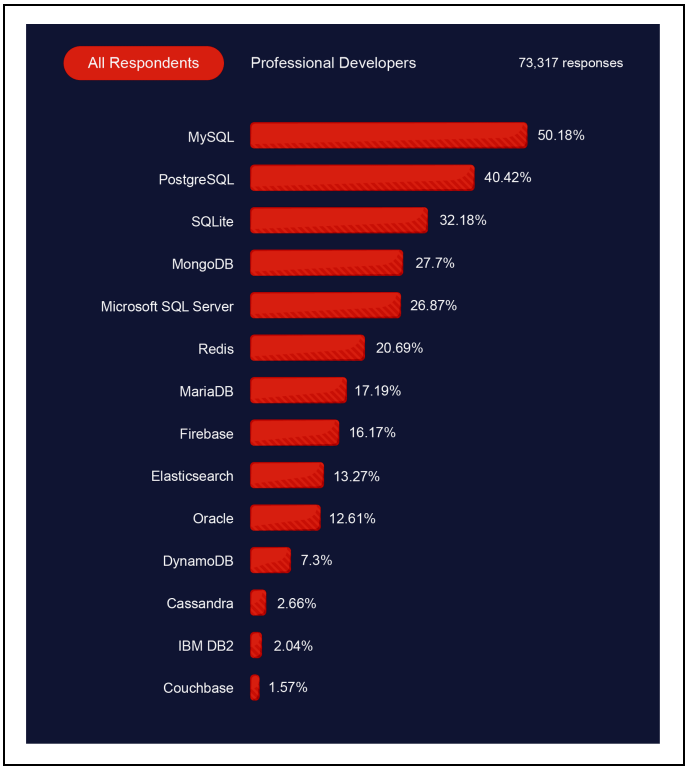
Full Extraction From MySQL Database

Understand full extraction methods from MySQL databases by running complete queries to pull all data from tables. Explore practical examples using SQL queries and Python scripting to export data for reporting, backups, and migration in ETL pipelines.

Introducing MySQL

The most common source of data for ETL pipelines is, to nobody's surprise, databases. According to ~74,000 responses on StackOverflow, the two most popular databases are MySQL and Postgres.



Ask

Stack Overflow survey for most popular databases, 2022

MySQL is an open-source OLTP relational database used by organizations worldwide. It's a simple database that's fast, reliable, well-understood, and easy to set up and manage. MySQL is known to be light on features to increase its speed and reliability. It's used for a wide range of applications, although the most popular usage is for web applications. MySQL is managed by Oracle and has a paid enterprise version and a free community edition.

MySQL is ACID (atomicity, consistency, isolation, and durability) compliant and has several storage engines that can be used to optimize the performance and customize it to our needs. The most popular ones are InnoDB and MyISAM. Many large organizations use MySQL. Some examples are:

- Google
- Facebook
- Twitter
- Spotify
- Booking.com
- GitHub, and many more...

Data extraction strategies

There are three methods for extracting data from MySQL.

- Full extraction
- Incremental extraction
- Extraction from MySQL's binary log

Extracting data using either full or incremental extraction is by far easier th third method, but it's also less scalable. Extracting from MySQL's binary log is more scalable but also more complex. It's suitable for large data volumes and

Ask

constantly changing data. It's also a good way of extracting streams of data instead of batches, which can be crucial depending on the application.

Full extraction

As the name suggests, full extraction of data means pulling all of the data that resides in a table or entire database. We don't choose what parts of the database to pull according to some business logic. We extract it as it is from the source. This is as easy as running a query like this `select * from <TableName>`.

Since we pull all of the currently available data in the source system, there's no need to keep track of changes to the data source since the last successful extraction. That makes full extraction by far the easiest method. Here are a few use cases for using full extraction:

1. Creating a backup for the source system
2. Generating reports
3. Data migration (for example, full extraction when moving from one database system to another)
4. Lastly, when we're unsure which parts of the data are relevant to the destination, we reluctantly might have to load everything

> **Note:** This method is easy to implement, but it will cause the heaviest overhead on the system. So we should only do this when we think the system can handle it

Examples

We can choose to extract data from a database such as MySQL using SQL queries or a programming language such as Python. For full and incremen extractions from MySQL, the recommended method is to use SQL queries to

Ask

extract the relevant data and export it as CSV. However, we'll explore how to do it using both methods.

Full extraction using SQL queries

We're tasked with preparing an extensive report on all past customers, regardless of their current status. Since our company is relatively small, we decided to extract all the customers' data.

To complete the task, we need to fetch all the data from the `customers` table and export it as CSV.

main.sql MySQL

```
-- Create table customers
CREATE TABLE customers (
  id int not null primary key,
  first_name varchar(30),
  last_name varchar(30),
  active int,
  LastUpdated timestamp
);

-- Populate table
INSERT INTO customers
  VALUES(1, 'Bruce', 'Wayne', 1, '2022-06-01 12:00:00');
INSERT INTO customers
```

Run

Full extraction using SQL

Line 27–29: We perform a full extraction on the `customers` table and export the query results as CSV to the current working directory.

As we can see, MySQL is raising an error. By default, MySQL will only let us export files into a secured file location. That secured location is stored in a system variable called `secure_file_priv`. Run the following code snippet to find the location.

Ask

main.sql MySQL

```
-- Show the address of MySQL's secured file location
SHOW VARIABLES LIKE "secure_file_priv";
```

Run

Finding MySQL's secured file location

The secured location for exporting files is: `/var/lib/mysql-files/`. Let's modify the code and export the `customers` table to that secured location.

main.sql MySQL

```
-- Create table customers
CREATE TABLE customers (
  id int not null primary key,
  first_name varchar(30),
  last_name varchar(30),
  active int,
  LastUpdated timestamp
);

-- Populate table
INSERT INTO customers
  VALUES(1, 'Bruce', 'Wayne', 1, '2022-06-01 12:00:00');
INSERT INTO customers
```

Ask

Exporting to secured file location

Line 27–29: We perform a full extraction on the `customers` table and export the query results as CSV to the location: `/var/lib/mysql-files/customers.csv`.

Full extraction using Python

Next, let's see how to perform a full extraction using Python. We'll use the driver `pymysql` to connect Python to MySQL and extract the data. To use the driver, we need to collect the connection details of the active MySQL server.

The required details are as follows:

- `host`
- `port`
- `user`
- `password`
- `database`

main.py Python 3.10.4

```
import pymysql
import csv

# Connect to the database
conn = pymysql.connect(
  host='localhost', port = 3306, database='marketing_dep',
  user='root', password='root')

# Create a cursor
cur = conn.cursor()

# Execute a SQL query
cur.execute("SELECT * FROM customers;")
```

Run

Ask

Full extraction using Python

- **Lines 5–7**: We use the `pymysql` driver to connect to MySQL. We specify the connection details of the active database.
- **Line 10:** We create a cursor object. We'll use it to pass commands to the database.
- **Line 13**: We use the cursor object to execute a query.
- **Line 16**: We collect results.

Ask